

# FlowLens

Plataforma Distribuida de Simulación Bancaria

## Banco A - Documentación Técnica

Versión 1.0 · 15 de April de 2026

### Contenido de este documento

1. Descripción general del proyecto
2. Arquitectura y estructura de carpetas
3. Base de datos y configuración
4. Modelos de datos (SQLAlchemy)
5. Esquemas de validación (Pydantic)
6. Seguridad y autenticación JWT
7. Capa de servicios (lógica de negocio)
8. Rutas de la API (endpoints)
9. Aplicación principal (main.py)
10. Frontend React - estructura y páginas
11. Cómo ejecutar el proyecto
12. Referencia de endpoints

## 1. Descripción general del proyecto

FlowLens es una plataforma distribuida de simulación bancaria y análisis transaccional. Su objetivo es reproducir el funcionamiento de múltiples bancos independientes con backends, bases de datos y usuarios propios, permitiendo además comunicación interbancaria y análisis externo de patrones sospechosos mediante un módulo de analytics.

Este documento cubre exclusivamente el Banco A (código BKA), que es la primera unidad funcional del sistema. Está construido con FastAPI, SQLAlchemy y SQLite, y expone una API REST completa para registro de usuarios, gestión de cuentas y transferencias internas.

### Alcance del Banco A

- Registro e inicio de sesión de usuarios con JWT y bcrypt.
- Creación de cuentas bancarias con número único generado automáticamente (formato BKA-XXXXXXXXXX).
- Consulta de saldo e historial de transacciones por usuario.
- Transferencias internas entre cuentas del mismo banco con validación de reglas de negocio.
- API documentada automáticamente en /docs (Swagger UI) y /redoc.

### Lo que NO incluye esta fase

- Transferencias interbancarias (fase futura).
- Motor de análisis de riesgo ni alertas (vive en el módulo analytics).
- Grafo de red transaccional.
- Despliegue en producción (diseñado para desarrollo local).

### Nota de seguridad

FlowLens es una simulación técnica demostrativa. No usa dinero real, no se conecta a sistemas bancarios reales y trabaja exclusivamente con datos sintéticos o de prueba. El módulo analytics requiere autenticación separada.

## 2. Arquitectura y estructura de carpetas

El Banco A sigue una arquitectura en capas limpia, donde cada directorio tiene una responsabilidad única. Esto facilita el mantenimiento, las pruebas y la extensión futura.

```

banco_a/
+-- app/
|   +-- __init__.py
|   +-- main.py           <- FastAPI app, CORS, registro de routers
|   +-- settings.py       <- Configuración global (env vars, constantes)
|   +-- database.py       <- Engine SQLAlchemy, Base, get_db, init_db
|   +-- core/
|       +-- security.py   <- JWT + bcrypt
|       +-- dependencies.py <- get_current_user (inyectable)
|   +-- models/
|       +-- user.py       <- Tabla users
|       +-- account.py    <- Tabla accounts
|       +-- transaction.py <- Tabla transactions
|   +-- schemas/
|       +-- user.py       <- Pydantic: UserCreate, UserResponse, etc.
|       +-- account.py    <- AccountCreate, AccountResponse
|       +-- transaction.py <- TransferRequest, TransactionResponse
|   +-- services/
|       +-- auth_service.py <- Registro y autenticación
|       +-- account_service.py <- Creación y consulta de cuentas
|       +-- transaction_service.py <- Transferencias y validaciones
|   +-- routes/
|       +-- auth.py       <- POST /auth/register, login; GET /auth/me
|       +-- accounts.py   <- POST /accounts; GET /accounts/my, /{id}
|       +-- transactions.py <- POST /transactions/internal; GET /transactions/my
+-- frontend/            <- Portal React del cliente
+-- requirements.txt
+-- run.py                <- Lanza uvicorn en el puerto 8000

```

### Flujo de una solicitud HTTP

Cuando un cliente realiza una petición, por ejemplo POST /transactions/internal, el flujo es el siguiente:

- FastAPI (main.py) recibe la petición y la enruta a transactions.router.
- El endpoint en routes/transactions.py valida el body con Pydantic (TransferRequest).
- La dependencia get\_current\_user (core/dependencies.py) verifica el JWT del header.
- El endpoint llama a transaction\_service.create\_internal\_transfer(db, data, user).
- El servicio aplica las reglas de negocio: monto > 0, cuentas distintas, saldo suficiente.
- Si todo es válido, actualiza los saldos y persiste la Transaction en SQLite.
- La respuesta se serializa con TransactionResponse (Pydantic) y se devuelve al cliente.

### Principio de separación de responsabilidades

| Capa      | Archivo              | Responsabilidad                                         |
|-----------|----------------------|---------------------------------------------------------|
| Config    | settings.py          | Centraliza todas las constantes y variables de entorno. |
| DB        | database.py          | Gestiona la conexión, Base y ciclo de vida de sesiones. |
| Seguridad | core/security.py     | Hash de contraseñas y generación/verificación de JWT.   |
| Dep.      | core/dependencies.py | Inyecta el usuario autenticado en los endpoints.        |

|           |               |                                                        |
|-----------|---------------|--------------------------------------------------------|
| Modelos   | models/*.py   | Define las tablas SQL como clases Python (ORM).        |
| Esquemas  | schemas/*.py  | Valida y serializa datos de entrada/salida (Pydantic). |
| Servicios | services/*.py | Contiene la lógica de negocio, sin acoplarse a HTTP.   |
| Rutas     | routes/*.py   | Declara los endpoints; delega lógica a servicios.      |

### 3. Base de datos y configuración

#### app/settings.py - Configuración global

Centraliza todas las constantes del banco. Todas leen variables de entorno con un valor por defecto, facilitando el despliegue en distintos entornos sin tocar el código.

```
BANK_ID      = os.getenv('BANK_ID',    'banco_a')
BANK_NAME    = os.getenv('BANK_NAME',  'Banco A')
BANK_CODE    = os.getenv('BANK_CODE',  'BKA')      # prefijo de números de cuenta
SECRET_KEY   = os.getenv('BANK_SECRET_KEY', '...') # clave para firmar JWT
ALGORITHM    = 'HS256'
ACCESS_TOKEN_EXPIRE_MINUTES = 720                # 12 horas
DATABASE_URL = f'sqlite:///{{DATABASE_PATH}}'
```

- BANK\_CODE se usa al generar números de cuenta: BKA-XXXXXXXXXX.
- SECRET\_KEY DEBE cambiarse en producción. El valor por defecto es solo para desarrollo.
- ALLOWED\_ORIGINS define los orígenes CORS permitidos (puertos de desarrollo de Vite).

#### app/database.py - Conexión y ciclo de vida

Este archivo es el núcleo de la persistencia. Define tres piezas fundamentales:

- engine: la conexión real a SQLite. Se crea una sola vez al importar el módulo.
- SessionLocal: fábrica de sesiones. Cada request HTTP abre una sesión nueva y la cierra al terminar.
- Base: clase padre de todos los modelos. Al heredar de Base, SQLAlchemy registra la clase en metadata.

```
def get_db():
    db = SessionLocal() # Dependencia inyectable en FastAPI
    # Abre sesión
    try:
        yield db # La cede al endpoint
    finally:
        db.close() # La cierra siempre, haya error o no

def init_db():
    # Llamada al arrancar la app
    from . import models # Importa los modelos (efecto de lado: los registra en Base)
    Base.metadata.create_all(bind=engine) # Crea las tablas físicas en SQLite
```

El hook PRAGMA foreign\_keys=ON activa la integridad referencial de SQLite, que por defecto está desactivada. Se dispara automáticamente en cada nueva conexión.

## 4. Modelos de datos (SQLAlchemy ORM)

Los modelos son clases Python que representan tablas en la base de datos. SQLAlchemy los mapea automáticamente. Cada columna es un atributo de clase.

### models/user.py - Tabla users

```
class User(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True)
    full_name = Column(String(255), nullable=False)
    email = Column(String(255), unique=True, index=True)
    hashed_password = Column(String(255), nullable=False)
    role = Column(String(50), default='client') # 'client' | 'admin'
    created_at = Column(DateTime, default=datetime.utcnow)
    accounts = relationship('Account', back_populates='owner', cascade='all, delete-orphan')
```

- email tiene un índice único - ningún usuario puede registrarse dos veces con el mismo correo.
- role está preparado para diferenciar clientes de administradores en fases futuras.
- La relación accounts permite acceder a User.accounts directamente desde Python.
- cascade='all, delete-orphan' borra las cuentas si el usuario es eliminado.

### models/account.py - Tabla accounts

```
class Account(Base):
    __tablename__ = 'accounts'
    id = Column(Integer, primary_key=True)
    account_number = Column(String(20), unique=True, index=True) # BKA-XXXXXXXXXX
    user_id = Column(Integer, ForeignKey('users.id'), index=True)
    bank_code = Column(String(10), default='BKA')
    balance = Column(Float, default=0.0)
    currency = Column(String(10), default='CRC')
    status = Column(String(20), default='active') # 'active' | 'inactive'
    created_at = Column(DateTime, default=datetime.utcnow)
    owner = relationship('User', back_populates='accounts')
```

- account\_number es único en toda la base de datos.
- bank\_code='BKA' identifica a qué banco pertenece la cuenta. Clave para futuros interbancarios.
- currency='CRC' (Colón costarricense) es el valor por defecto.
- status permite desactivar cuentas sin eliminarlas.

### models/transaction.py - Tabla transactions

```
class Transaction(Base):
    __tablename__ = 'transactions'
    id = Column(Integer, primary_key=True)
    source_account_number = Column(String(20), index=True)
    destination_account_number = Column(String(20), index=True)
    source_bank_code = Column(String(10))
    destination_bank_code = Column(String(10))
    amount = Column(Float)
```



`transaction_type``= Column(String(50), default='internal_transfer')`



status



location

= Column(String(255), nullable=True)

description

= Column(Text, nullable=True)

created\_at

= Column(DateTime, default=datetime.utcnow)

Decisión de diseño clave: Transaction usa account\_number (strings) en lugar de FKs enteras a la tabla accounts. Esto permite registrar transacciones con cuentas externas (de otros bancos) sin violar integridad referencial. Es esencial para soportar transferencias interbancarias en fases futuras.

## 5. Esquemas de validación (Pydantic)

Los schemas son la capa de contrato entre el cliente HTTP y la aplicación. Pydantic valida automáticamente los tipos, formatos y restricciones antes de que el código de negocio siquiera se ejecute. Errores de formato retornan 422 Unprocessable Entity.

### [schemas/user.py](#)

```
class UserCreate(BaseModel):    # Body del POST /auth/register
    full_name: str
    email: EmailStr            # Valida que sea un email real
    password: str

class UserResponse(BaseModel): # Lo que devuelve la API (nunca expone hashed_password)
    id: int | full_name: str | email: str | role: str | created_at: datetime
    model_config = {'from_attributes': True} # Permite leer desde objetos SQLAlchemy

class LoginRequest(BaseModel): # Body del POST /auth/login
    email: EmailStr | password: str

class TokenResponse(BaseModel): # Respuesta del login
    access_token: str | token_type: str = 'bearer' | user: UserResponse
```

### [schemas/account.py](#)

```
class AccountCreate(BaseModel): # Body del POST /accounts
    initial_balance: float = 0.0

class AccountResponse(BaseModel):
    id, account_number, bank_code, balance, currency, status, created_at
```

### [schemas/transaction.py](#)

```
class TransferRequest(BaseModel): # Body del POST /transactions/internal
    source_account_number: str
    destination_account_number: str
    amount: float
    channel: str = 'web'         # web | mobile | atm | api
    location: str = ''
    description: str = ''

class TransactionResponse(BaseModel):
    id, source_account_number, destination_account_number,
    source_bank_code, destination_bank_code, amount, currency,
    transaction_type, status, channel, location, description, created_at
```

## 6. Seguridad y autenticación JWT

### core/security.py - Hash y tokens

Este módulo encapsula toda la criptografía. Usa dos librerías:

- passlib[bcrypt]: para hashear y verificar contraseñas. bcrypt es el estándar de la industria para almacenamiento seguro de passwords. Jamás se guarda la contraseña en texto plano.
- python-jose: para generar y decodificar tokens JWT firmados con HS256.

```
def hash_password(password: str) -> str:
    return pwd_context.hash(password)          # Retorna '$2b$12$...' (bcrypt)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)    # Compara sin exponer el hash

def create_access_token(data: dict) -> str:
    payload = data.copy()
    payload['exp'] = datetime.utcnow() + timedelta(minutes=720) # 12 horas
    return jwt.encode(payload, SECRET_KEY, algorithm='HS256')

def decode_access_token(token: str) -> dict | None:
    try: return jwt.decode(token, SECRET_KEY, algorithms=['HS256'])
    except JWTError: return None # Token inválido o expirado
```

### core/dependencies.py - get\_current\_user

Esta es la dependencia de autenticación que se inyecta en todos los endpoints protegidos. FastAPI la llama automáticamente gracias al decorador Depends().

```
oauth2_scheme = OAuth2PasswordBearer(tokenUrl='/auth/login')

def get_current_user(token = Depends(oauth2_scheme), db = Depends(get_db)) -> User:
    payload = decode_access_token(token)
    if payload is None:
        raise HTTPException(401, 'Token inválido o expirado')
    user = db.query(User).filter(User.id == int(payload['sub'])).first()
    if user is None:
        raise HTTPException(401, 'Usuario no encontrado')
    return user # FastAPI lo inyecta en el parámetro del endpoint
```

- El token llega en el header: Authorization: Bearer <token>
- payload['sub'] contiene el user\_id como string (estándar JWT).
- Si el token expiró o es inválido, retorna HTTP 401 automáticamente.
- Los endpoints protegidos solo añaden: current\_user: User = Depends(get\_current\_user)



## 7. Capa de servicios (lógica de negocio)

Los servicios son el corazón de la aplicación. Contienen la lógica de negocio completamente desacoplada del protocolo HTTP. Esto significa que la misma lógica podría ser llamada desde una CLI, un test unitario o un worker sin cambiar nada.

### services/auth\_service.py

- register\_user(db, data): verifica que el email no exista, hashea la contraseña y crea el usuario.
- authenticate\_user(db, email, password): busca al usuario, verifica la contraseña con bcrypt y devuelve (token, user). Si falla, lanza HTTP 401.

### services/account\_service.py

- \_generate\_account\_number(db): genera un número BKA-XXXXXXXXXX con dígitos aleatorios, verificando en la base de datos que no exista ya (loop hasta encontrar uno único).
- create\_account(db, user, initial\_balance): crea la cuenta con bank\_code='BKA' y currency='CRC'.
- get\_accounts\_by\_user(db, user): retorna todas las cuentas del usuario autenticado.
- get\_account\_by\_id(db, account\_id, user): retorna una cuenta específica, validando propiedad.

### services/transaction\_service.py - Reglas de negocio

Este es el servicio más importante. Implementa todas las validaciones antes de mover dinero:

- Monto > 0: no se puede transferir 0 ni montos negativos.
- Cuenta origen - destino: no se puede transferir a sí mismo.
- Cuentas deben existir y estar activas (status='active').
- El usuario autenticado debe ser propietario de la cuenta origen.
- Ambas cuentas deben pertenecer al banco BKA (transferencias internas).
- Saldo suficiente: source.balance >= amount.

```
# Si todas las validaciones pasan:
source.balance -= data.amount      # Débito
destination.balance += data.amount # Crédito
txn = Transaction(...)             # Registro inmutable
db.add(txn)
db.commit()                        # Atómico: todo o nada
```

El commit es atómico: si ocurre cualquier error después del débito pero antes del crédito, SQLAlchemy hace rollback automáticamente y ninguna cuenta queda descuadrada.

## 8. Rutas de la API (endpoints)

Los routers de FastAPI definen los endpoints HTTP. Son deliberadamente delgados: validan la entrada con Pydantic, inyectan dependencias y delegan la lógica al servicio.

### routes/auth.py

```
# POST /auth/register -> 201 Created
# Body: { full_name, email, password }
# Retorna: UserResponse

# POST /auth/login -> 200 OK
# Body: { email, password }
# Retorna: { access_token, token_type, user }

# GET /auth/me -> 200 OK [PROTEGIDO]
# Header: Authorization: Bearer <token>
# Retorna: UserResponse del usuario autenticado
```

### routes/accounts.py

```
# POST /accounts -> 201 Created [PROTEGIDO]
# Body: { initial_balance: 0.0 }
# Retorna: AccountResponse con número de cuenta generado

# GET /accounts/my -> 200 OK [PROTEGIDO]
# Retorna: lista de AccountResponse del usuario autenticado

# GET /accounts/{account_id} -> 200 OK [PROTEGIDO]
# Retorna: AccountResponse (solo si pertenece al usuario autenticado)
```

### routes/transactions.py

```
# POST /transactions/internal -> 201 Created [PROTEGIDO]
# Body: { source_account_number, destination_account_number,
#        amount, channel, location, description }
# Retorna: TransactionResponse

# GET /transactions/my -> 200 OK [PROTEGIDO]
# Retorna: todas las transacciones donde el usuario es origen o destino
#          ordenadas por fecha descendente
```

### app/main.py - Punto de entrada de FastAPI

```
app = FastAPI(title='Banco A API', lifespan=lifespan)

# lifespan llama a init_db() al arrancar -> crea tablas en SQLite
```

```
app.add_middleware(CORSMiddleware, allow_origins=ALLOWED_ORIGINS, ...)
```



```
app.include_router(auth.router) # prefix='/auth'
```

```
app.include_router(accounts.router) # prefix='/accounts'
```

```
app.include_router(transactions.router) # prefix='/transactions'
```





```
@app.get('/') # Health check: { status: 'ok', bank: 'Banco A' }
```

## 9. Aplicación principal (main.py)

main.py es el archivo que FastAPI ejecuta. Tiene tres responsabilidades:

- Declarar la aplicación con FastAPI() y sus metadatos (título, versión, descripción).
- Configurar el middleware CORS para que el frontend pueda hacer peticiones.
- Registrar los tres routers (auth, accounts, transactions) con sus prefijos.

### El patrón lifespan

FastAPI moderno usa un context manager asíncrono (lifespan) para el ciclo de vida. Es más limpio que los viejos eventos `@app.on_event('startup')`.

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    init_db()    # <- Se ejecuta ANTES de recibir el primer request
    yield        # <- La app corre aquí
                # Después del yield: cleanup (si hubiera)
```

### CORS - Cross-Origin Resource Sharing

CORS es el mecanismo que permite al navegador hacer peticiones desde un origen (`http://localhost:5173` - Vite) a otro (`http://127.0.0.1:8000` - FastAPI). Sin este middleware, el navegador bloquea todas las respuestas del backend. `allow_credentials=True` es necesario para enviar el header `Authorization`.

## 10. Frontend React - estructura y páginas

El frontend es una Single Page Application (SPA) construida con React 18 y Vite. Se comunica exclusivamente con el backend a través de la API REST. No tiene acceso directo a la base de datos.

### Archivos clave

- src/config.js: exporta constantes leídas de variables de entorno Vite (VITE\_API\_URL, etc.).
- src/api.js: función apiRequest() que envuelve fetch con manejo de tokens y errores. Incluye formatCurrency() (Colón costarricense) y formatDate() (locale es-CR).
- src/auth.jsx: AuthContext y AuthProvider. Gestiona token y usuario en localStorage. Valida el token contra /auth/me al montar la app.
- src/styles.css: tema azul profesional con variables CSS, layout de sidebar fijo + contenido.
- src/App.jsx: enrutamiento con react-router-dom. ProtectedRoute redirige a /login si no hay sesión.

### Páginas

| Página           | Ruta          | Descripción                                                               |
|------------------|---------------|---------------------------------------------------------------------------|
| LoginPage        | /login        | Formulario email/password. Guarda token en localStorage.                  |
| RegisterPage     | /register     | Crea usuario y hace login automático al terminar.                         |
| DashboardPage    | /             | Resumen: saldo total, cuentas, enviado/recibido, últimas 8 txns.          |
| AccountsPage     | /accounts     | Crea cuentas con saldo inicial. Tabla de todas las cuentas.               |
| TransferPage     | /transfer     | Selecciona cuenta origen, ingresa destino y monto. Valida en tiempo real. |
| TransactionsPage | /transactions | Historial completo. Verde=recibida, Rojo=enviada. Resumen de totales.     |

### Flujo de autenticación en el frontend

- Al iniciar sesión, el servidor retorna { access\_token, user }.
- AuthProvider guarda ambos en localStorage (bka\_token, bka\_user).
- Al montar la app, llama GET /auth/me para verificar que el token siga válido.
- Si el token expiró o es inválido, limpia localStorage y redirige a /login.
- Todos los componentes acceden al contexto via el hook useAuth().

## 11. Cómo ejecutar el proyecto

### Requisitos previos

- Python 3.10 o superior.
- Node.js 18 o superior + npm.
- Un entorno virtual Python (recomendado).

### Backend (Banco A)

```
# 1. Ir al directorio del banco
cd banco_a

# 2. Crear y activar entorno virtual
python -m venv .venv
source .venv/bin/activate      # Mac/Linux
.venv\Scripts\activate        # Windows

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Levantar la API
python run.py

# La API queda disponible en:
# http://127.0.0.1:8000
# Swagger UI: http://127.0.0.1:8000/docs
# ReDoc:      http://127.0.0.1:8000/redoc
```

### Frontend

```
# 1. Ir al directorio del frontend
cd banco_a/frontend

# 2. Instalar dependencias
npm install

# 3. Levantar el servidor de desarrollo
npm run dev

# El portal queda en: http://localhost:5173
```

### Primera prueba rápida

- Abrir <http://127.0.0.1:8000/docs>
- Usar POST /auth/register para crear un usuario.
- Usar POST /auth/login para obtener el token. Copiar access\_token.
- Hacer clic en 'Authorize' (candado) y pegar el token.
- Usar POST /accounts para crear una cuenta con saldo inicial.
- Crear una segunda cuenta.

- Usar POST /transactions/internal para transferir entre las dos cuentas.
- Verificar con GET /transactions/my que la transacción fue registrada.

## Variables de entorno opcionales

---

```
BANK_ID=banco_a
BANK_NAME='Banco A'
BANK_CODE=BKA
BANK_SECRET_KEY=mi-clave-super-secreta-2026
BANK_DB_PATH=/ruta/absoluta/a/banco_a.db
ACCESS_TOKEN_EXPIRE_MINUTES=720
VITE_API_URL=http://127.0.0.1:8000 # frontend
```

## 12. Referencia completa de endpoints

| Método | Endpoint               | Auth | Descripción                                                    |
|--------|------------------------|------|----------------------------------------------------------------|
| GET    | /                      | No   | Health check. Retorna status, banco y banco_id.                |
| POST   | /auth/register         | No   | Registra un nuevo usuario. Body: {full_name, email, password}. |
| POST   | /auth/login            | No   | Login. Body: {email, password}. Retorna access_token + user.   |
| GET    | /auth/me               | Sí   | Retorna el usuario autenticado actualmente.                    |
| POST   | /accounts              | Sí   | Crea una cuenta bancaria. Body: {initial_balance}.             |
| GET    | /accounts/my           | Sí   | Lista todas las cuentas del usuario autenticado.               |
| GET    | /accounts/{account_id} | Sí   | Retorna una cuenta por ID (solo si es del usuario).            |
| POST   | /transactions/internal | Sí   | Transferencia interna. Body: {source, dest, amount, ...}.      |
| GET    | /transactions/my       | Sí   | Historial de transacciones del usuario autenticado.            |

### Códigos de error más comunes

| Código HTTP              | Significado                                                    |
|--------------------------|----------------------------------------------------------------|
| 400 Bad Request          | Datos inválidos: monto <= 0, misma cuenta origen/destino, etc. |
| 401 Unauthorized         | Token ausente, inválido o expirado. Credenciales incorrectas.  |
| 403 Forbidden            | El usuario no es propietario de la cuenta origen.              |
| 404 Not Found            | Cuenta no encontrada o inactiva.                               |
| 422 Unprocessable Entity | Error de validación Pydantic. El body no cumple el schema.     |

### Próximas fases de FlowLens

Fase 2: Transferencias interbancarias entre Banco A, Banco B y Banco C.

Fase 3: Módulo externo de Analytics con motor de riesgo por reglas, alertas y grafo de red transaccional.

Fase 4: Dashboard de analista con seguimiento profundo de cuentas sospechosas.

Fase 5: Despliegue en contenedores con Docker Compose.